PROJECT BLUEPRINT

Banking Transaction Simulation System

Fraud Detection + Backend Engineering + DSA

For Freshers | College Placement Edition

3-4 Week Build Plan • Node.js + PostgreSQL + HTML/CSS/JS • Free Deployment

1. Project Overview

You are building a backend-heavy banking system that shows you understand authentication, database transactions, security, and algorithmic thinking — all in one project. This is not just a CRUD app. The DSA-based fraud detection layer is your biggest differentiator from every other fresher project.

What This Project Proves to a Recruiter

Skill Area	What the Recruiter Sees
Backend Engineering	Can you design a layered, clean backend architecture?
Database Skills	Do you understand ACID transactions and schema design?
Security Awareness	JWT, bcrypt, rate limiting — do you think about auth properly?
DSA in Practice	Can you use algorithms to solve real problems, not just LeetCode?
System Thinking	Can you build something end-to-end and explain it confidently?

2. Tech Stack (Final, Simplified)

Layer	Technology	Why This Choice

Backend Runtime	Node.js + Express	Widely used, easy to explain, great for REST APIs
Database	PostgreSQL	ACID compliant, perfect for financial data, free tier on Render
Authentication	JWT + bcrypt	Industry standard — every interviewer will ask about this
Frontend	HTML + CSS + JavaScript	No frameworks needed, keeps focus on backend
Deployment	Render.com (free)	Free tier for both Node.js backend and PostgreSQL DB
API Testing	Postman	Export your collection — it becomes part of your submission
Version Control	GitHub	Public repo with clean README is your portfolio link

What we are NOT using: Redis (caching), Winston (advanced logging), Docker, TypeScript, React. These add complexity with no interview benefit at the fresher level. Keep it simple and explained well.

3. Folder Structure

This is how your project should look when complete. Build it exactly this way from day one.

```
banking-system/
├── backend/
│   ├── src/
│   │   ├── routes/          ← API endpoint definitions
│   │   ├── controllers/     ← Handle req/res, call services
│   │   ├── services/        ← All business logic lives here
│   │   ├── models/          ← Database queries
│   │   ├── middlewares/     ← Auth check, rate limit, validation
│   │   ├── fraud/           ← Your 3 DSA fraud modules
│   │   └── utils/           ← JWT helper, password helper
│   ├── schema.sql           ← Your DB schema (very important!)
│   ├── app.js               ← Express app setup
│   ├── server.js            ← Entry point
│   └── .env.example         ← Never commit .env itself
├── frontend/
│   ├── index.html           ← Login / Register page
│   ├── dashboard.html       ← Account + transfer UI
```

```
│   ├── admin.html           ← Fraud alerts + flagged users
│   ├── style.css
│   └── app.js
├── README.md                ← Critical. Write this well.
└── postman_collection.json  ← Export from Postman when done
```

4. Database Schema

This is the first thing an interviewer asks about. Know this cold. Four tables, that is all you need.

```sql
-- Users Table
CREATE TABLE users (
  id          SERIAL PRIMARY KEY,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(100) UNIQUE NOT NULL,
  password    VARCHAR(255) NOT NULL,   -- bcrypt hash
  role        VARCHAR(10) DEFAULT 'user',  -- 'user' or 'admin'
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Accounts Table (one per user for simplicity)
CREATE TABLE accounts (
  id          SERIAL PRIMARY KEY,
  user_id     INT REFERENCES users(id),
  balance     NUMERIC(15,2) DEFAULT 0.00,
  created_at  TIMESTAMP DEFAULT NOW()
);

-- Transactions Table (audit trail lives here)
CREATE TABLE transactions (
  id              SERIAL PRIMARY KEY,
  sender_id       INT REFERENCES accounts(id),
  receiver_id     INT REFERENCES accounts(id),
  amount          NUMERIC(15,2) NOT NULL,
  type            VARCHAR(20),  -- 'deposit','withdraw','transfer'
  status          VARCHAR(20) DEFAULT 'completed',
  created_at      TIMESTAMP DEFAULT NOW()
);

-- Fraud Flags Table
CREATE TABLE fraud_flags (
  id          SERIAL PRIMARY KEY,
  user_id     INT REFERENCES users(id),
  reason      TEXT,
  flagged_at  TIMESTAMP DEFAULT NOW()
);
```

5. API Endpoints Reference

Auth Endpoints

Method	Endpoint	What it does
POST	/api/auth/register	Register a new user with hashed password
POST	/api/auth/login	Login, returns JWT token

Account Endpoints (JWT Protected)

Method	Endpoint	What it does
POST	/api/accounts	Create a bank account for logged-in user
GET	/api/accounts/me	Get your account balance and details
GET	/api/accounts/transactions	Get your full transaction history
POST	/api/transactions/deposit	Deposit money into your account
POST	/api/transactions/withdraw	Withdraw money from your account
POST	/api/transactions/transfer	Transfer money to another account

Admin Endpoints (Admin Role Required)

Method	Endpoint	What it does
GET	/api/admin/fraud/flags	See all fraud flagged transactions
GET	/api/admin/fraud/top-suspicious	Top K users with most flags (heap)
GET	/api/admin/users/ranking	All users ranked by activity

6. Build Phases (Week by Week)

PHASE 1 Project Setup + Auth System — Days 1 - 5

Goal: By end of Week 1, someone should be able to register, login, and get back a JWT token. Nothing else. Foundation first.

Day 1 — Environment Setup

- Install Node.js (v18+), PostgreSQL, VS Code, Postman
- Create GitHub repo, push initial empty project

- Run: npm init -y in the backend folder
- Install packages: npm install express pg bcryptjs jsonwebtoken dotenv express-validator express-rate-limit cors
- Create your full folder structure as shown in Section 3
- Create your .env file with DB credentials and JWT secret

Day 2 — Database Setup

- Install PostgreSQL locally OR create a free Render PostgreSQL database right now
- Run the schema.sql file from Section 4 to create all 4 tables
- Test that you can connect to the DB from Node.js using the pg package
- Create a db.js file in utils/ that exports a query function

Day 3-4 — Auth Routes

- Start coding in this order: utils/jwt.js first, then utils/password.js, then models/userModel.js, then services/authService.js, then controllers/authController.js, then routes/authRoutes.js
- Register endpoint: take name, email, password. Hash the password with bcrypt. Save to DB. Return success message.
- Login endpoint: find user by email. Compare password with bcrypt.compare(). If match, sign and return JWT.

Day 5 — Auth Middleware + Testing

- Write middlewares/authMiddleware.js: extract JWT from Authorization header, verify it, attach user to req.user
- Write middlewares/roleMiddleware.js: check if req.user.role === 'admin'
- Test register and login in Postman. Save both requests to your collection.
- Push to GitHub with a meaningful commit message

PHASE 2 Account Management + Transactions (ACID) — Days 6 - 12

Goal: By end of this phase, users can create accounts, deposit, withdraw, and transfer money. This is your core banking logic and your most important interview talking point.

Day 6-7 — Account Creation + Balance

- models/accountModel.js: functions to create account, get account by user ID, get balance
- services/accountService.js: business logic — check if account already exists before creating
- controllers/accountController.js and routes/accountRoutes.js
- Test: create account for a registered user. Fetch account details. All behind JWT middleware.

Day 8-9 — Deposit and Withdraw (Your First ACID Transaction)

- This is critical. Do NOT just do a simple UPDATE. Use a proper DB transaction.
- Deposit logic in services/transactionService.js:
  - Step 1: BEGIN
  - Step 2: UPDATE accounts SET balance = balance + amount WHERE id = ?
  - Step 3: INSERT into transactions table with type='deposit'
  - Step 4: COMMIT — if anything fails, ROLLBACK
- Withdraw adds one check: if balance < amount, throw an error BEFORE beginning the transaction
- Test both in Postman. Try withdrawing more than balance — it should fail gracefully.

Day 10-11 — Transfer Funds (The Hard One)

- Transfer = debit sender + credit receiver + log transaction, all atomically
- Use SELECT FOR UPDATE on both accounts inside the transaction — this prevents race conditions and is your interview gold answer
- Full flow: BEGIN > lock both rows > check sender balance > debit sender > credit receiver > insert transaction record > COMMIT
- If accounts are same, return error. If sender balance insufficient, ROLLBACK.

Day 12 — Transaction History + Validation

- GET /api/accounts/transactions: return all transactions for logged-in user, ordered by latest first
- Add express-validator to all POST routes: check amount is a positive number, receiver account ID exists, etc.
- Test the full flow end to end in Postman. Export your Postman collection.

PHASE 3 Fraud Detection (DSA Modules) — Days 13 - 19

Goal: Implement all 3 fraud detection modules. These are your interview differentiators. Each one maps directly to a DSA concept. Be ready to explain the logic on a whiteboard.

Module 1 — Sliding Window: High Frequency Detection

DSA concept: Queue-based sliding window

Interview explanation: *"I maintain a queue of transaction timestamps per user. When a new transaction comes in, I remove all timestamps older than 60 seconds from the front of the queue, then add the current timestamp. If the queue size exceeds a threshold (e.g., 5 transactions), I flag the user."*

```
// fraud/slidingWindow.js
const userTransactionQueues = new Map();

function checkHighFrequency(userId, windowMs = 60000, maxCount = 5) {
  const now = Date.now();
```

```
  if (!userTransactionQueues.has(userId)) {
    userTransactionQueues.set(userId, []);
  }
  const queue = userTransactionQueues.get(userId);
  // Remove timestamps outside the window
  while (queue.length > 0 && now - queue[0] > windowMs) {
    queue.shift();
  }
  queue.push(now);
  return queue.length > maxCount;  // true = suspicious
}
```

Module 2 — Graph + DFS: Circular Transaction Detection

DSA concept: Directed graph with DFS cycle detection using visited set + recursion stack

Interview explanation: *"Each user is a node. Each transfer is a directed edge from sender to receiver. I run DFS from each unvisited node. If DFS encounters a node already in the current recursion stack, a cycle exists — meaning money is going in a circle, which is a money laundering pattern."*

```
// fraud/cycleDetection.js
function buildGraph(transactions) {
  const graph = {};
  for (const tx of transactions) {
    if (!graph[tx.sender_id]) graph[tx.sender_id] = [];
    graph[tx.sender_id].push(tx.receiver_id);
  }
  return graph;
}

function hasCycleDFS(node, graph, visited, recStack) {
  visited.add(node);
  recStack.add(node);
  for (const neighbor of (graph[node] || [])) {
    if (!visited.has(neighbor) && hasCycleDFS(neighbor, graph, visited, recStack))
      return true;
    if (recStack.has(neighbor)) return true;
  }
  recStack.delete(node);
  return false;
}

function detectCircularTransactions(transactions) {
  const graph = buildGraph(transactions);
  const visited = new Set();
  const recStack = new Set();
  for (const node of Object.keys(graph)) {
    if (!visited.has(node) && hasCycleDFS(node, graph, visited, recStack))
      return true;
  }
  return false;
}
```

Module 3 — Max Heap: Top Suspicious Users

DSA concept: Priority queue / max-heap to efficiently retrieve top K flagged users

Interview explanation: *"I maintain a count of fraud flags per user. To get the top K suspicious users, I use a max-heap (or a sorted array for simplicity) that orders users by flag count. This gives O(log n) insertion and O(1) retrieval of the most suspicious user."*

```
// fraud/suspiciousHeap.js
// Simplified max-heap using sorted array
class SuspiciousUsersHeap {
  constructor() { this.heap = []; }

  addOrUpdate(userId, flagCount) {
    const idx = this.heap.findIndex(u => u.userId === userId);
    if (idx !== -1) this.heap[idx].flagCount = flagCount;
    else this.heap.push({ userId, flagCount });
    this.heap.sort((a, b) => b.flagCount - a.flagCount); // max-heap behavior
  }

  getTopK(k) {
    return this.heap.slice(0, k);
  }
}

module.exports = new SuspiciousUsersHeap(); // singleton
```

Day 18-19 — Wire Fraud Modules Into the System

- Call checkHighFrequency() inside transactionService.js after every transaction. If flagged, insert a row into fraud_flags table.
- Run detectCircularTransactions() periodically (on admin request) by fetching recent transactions from DB and building the graph.
- Admin endpoint GET /api/admin/fraud/top-suspicious calls heap.getTopK(10) — query fraud_flags, count by user_id, feed into heap.
- Test by making many rapid transactions with one user and confirming they get flagged.

PHASE 4 Frontend + Deployment + Polish — Days 20 - 26

Goal: A working UI that talks to your backend, deployed live on Render with a clean README. This is what you share in interviews and on your resume.

Day 20-21 — Frontend (HTML/CSS/JS)

- index.html: Register form + Login form. On login success, store the JWT in localStorage. Redirect to dashboard.
- dashboard.html: Show account balance (fetch from /api/accounts/me). Deposit / Withdraw / Transfer forms. Transaction history table.

- admin.html: Only accessible if user role is admin. Show fraud flags table and top suspicious users.
- app.js: Use fetch() to call your backend. Always include Authorization: Bearer <token> header on protected calls.
- style.css: Keep it simple and clean. You are not a frontend developer — make it functional, not fancy.

Day 22-23 — Rate Limiting

- Add express-rate-limit to your login endpoint: maximum 5 attempts per 15 minutes per IP
- Add a separate rate limiter for transaction endpoints: maximum 10 transactions per minute per user
- This is a one-day task and shows security awareness to interviewers

Day 24-25 — Deployment on Render (Free)

- Step 1: Push your final code to GitHub
- Step 2: Go to render.com and create a free account
- Step 3: Create a new PostgreSQL database on Render. Copy the connection string.
- Step 4: Create a new Web Service on Render. Connect your GitHub repo. Set build command to npm install and start command to node server.js
- Step 5: Add your environment variables (DB URL, JWT secret) in Render's environment settings
- Step 6: Run your schema.sql on the Render DB using the psql command Render gives you
- Step 7: Test your live URL in Postman
- Step 8: Update your README with the live URL

Day 26 — Final Polish

- Write your README.md properly — see Section 8 below
- Export your Postman collection as a JSON file and commit it to the repo
- Clean up your code: remove all console.log debug statements, add short comments to fraud modules
- Test the entire flow end to end one more time on the live deployment
- Add your GitHub repo link and live URL to your resume

7. Must Build vs Skip List

Feature	Status	Reason
JWT Auth + bcrypt	**BUILD**	Core interview question
ACID fund transfers	**BUILD**	Your strongest technical differentiator
Sliding window fraud detection	**BUILD**	DSA + real use case combo

Graph cycle detection	**BUILD**	Most impressive algorithm in the project
Max heap suspicious users	**BUILD**	Shows priority queue knowledge
Transaction history + audit trail	**BUILD**	Shows DB design thinking
Admin role + fraud dashboard	**BUILD**	Completes the story, simple to add
Input validation	**BUILD**	Shows you think about security
Rate limiting on login	**BUILD**	Simple, one file, big impact
HTML/CSS/JS frontend	**BUILD**	Makes it demonstrable in interviews
README + Postman collection	**BUILD**	The most important non-code deliverable
Redis caching	**SKIP**	Adds complexity, no fresher benefit
Winston logging	**SKIP**	console.log is enough for placement level
Docker / containerisation	**SKIP**	Overkill, will confuse your setup
TypeScript	**SKIP**	Adds friction, not asked at fresher level
React frontend	**SKIP**	Focus is backend, plain JS is fine
Retry with exponential backoff	**SKIP**	Too advanced for this stage

8. README Template (Copy This)

Your README is your project's resume. Recruiters often look at this before the code. Write it like you are explaining the project to a non-technical person, then get more technical.

```
# Banking Transaction Simulation System

A production-style backend banking system with ACID-compliant fund transfers
and DSA-based fraud detection. Built with Node.js, Express, and PostgreSQL.

## Live Demo
https://your-render-url.onrender.com

## Features
- JWT authentication with bcrypt password hashing
- ACID-compliant deposit, withdraw, and transfer using DB transactions
- Fraud Detection:
  - Sliding window for high-frequency transaction alerts
  - Graph + DFS for circular/money-laundering pattern detection
  - Max-heap for top-K suspicious user retrieval
- Role-based access (User / Admin)
- Rate limiting on login and transaction endpoints
```

```
## Tech Stack
Node.js | Express | PostgreSQL | JWT | bcrypt

## Run Locally
git clone <repo-url>
cd backend && npm install
cp .env.example .env  # fill in your DB credentials
psql -U postgres -f schema.sql
node server.js

## API Docs
Import postman_collection.json into Postman to test all endpoints.
```

9. Interview Questions to Prepare

Every question below has been asked in real placement interviews. Know your answers before the demo day.

On Transactions and Database

- What is ACID? How does your project implement it?
- What happens if the server crashes in the middle of a transfer? (Answer: ROLLBACK happens automatically because COMMIT was never reached)
- What is a race condition? How did you prevent it? (Answer: SELECT FOR UPDATE locks the rows)
- Why did you use PostgreSQL over MongoDB here?

On Authentication

- Why JWT and not sessions?
- Where do you store the JWT on the client side?
- What is the difference between authentication and authorization?
- Why do you hash passwords and not encrypt them?

On the Fraud Detection Modules

- Walk me through the sliding window algorithm you implemented.
- How does DFS detect cycles in a graph?
- What is the time complexity of your cycle detection?
- Why a heap for top-K users? What is the alternative?

On Architecture

- Why did you separate routes, controllers, and services?

- What is the difference between your service layer and your model layer?
- How would you scale this system if 1 million users use it?

10. Your 60-Second Interview Pitch

Memorise and practise saying this out loud:

"I built a backend banking system in Node.js and PostgreSQL. The core feature is ACID-compliant fund transfers — I used database-level transactions with SELECT FOR UPDATE to prevent race conditions and ensure no double spending happens.

What makes it different from a normal CRUD project is the fraud detection layer. I implemented three algorithms: a sliding window to catch high-frequency transactions, a graph with DFS cycle detection to identify circular money flows, and a max-heap to efficiently retrieve the most suspicious users.

It's deployed live on Render and the full API is documented in Postman."

11. Final Submission Checklist

Deliverable	What Done Looks Like
Code	All 4 phases complete, no debug console.log, code is readable
schema.sql	Committed in the repo root, creates all 4 tables cleanly
README.md	Has live URL, setup steps, features list, tech stack
Postman Collection	All endpoints tested and exported as JSON in the repo
Frontend	Login, dashboard, admin panel all working against live API
Fraud Modules	All 3 modules wired into the system, flagging works
Deployment	Live URL on Render, accessible without login for the demo pages
GitHub	Clean commit history, meaningful commit messages, public repo
Interview Prep	Can explain every module, can answer all Section 9 questions
Resume	GitHub link and live demo URL both added to your resume